



LẬP TRÌNH GAME 3D NÂNG CAO

BÀI 2: Interpolation trong UNITY 3D

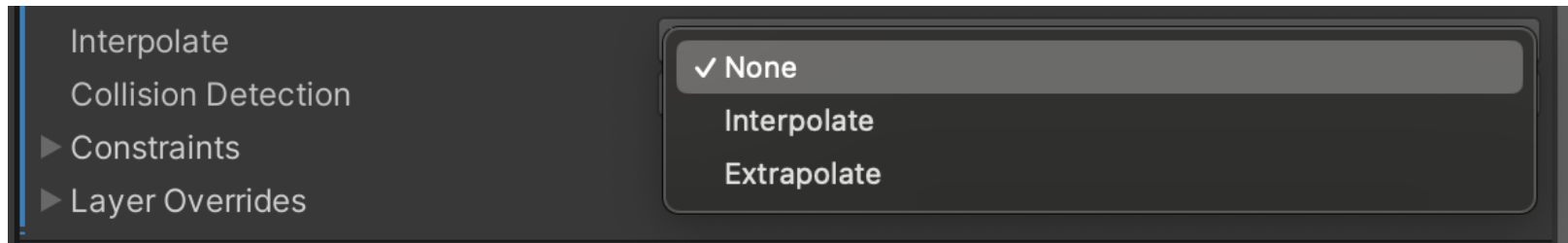
- ☐ Interpolation là gì ?
- ☐ Các chế độ Interpolation
- ☐ Tại sao sử dụng Interpolation
- ☐ FrameRate
- ☐ ConstantForce trong Unity

- ❑ **Interpolation** trong Unity được sử dụng để làm cho chuyển động của một đối tượng trông mượt mà hơn, đặc biệt là khi tốc độ khung hình (framerate) của trò chơi không ổn định.
- ❑ Khi Interpolation được bật, Unity sẽ tính toán vị trí hiển thị của Rigidbody giữa các khung hình dựa trên tốc độ di chuyển vật lý, giúp chuyển động trông tự nhiên và không bị giật

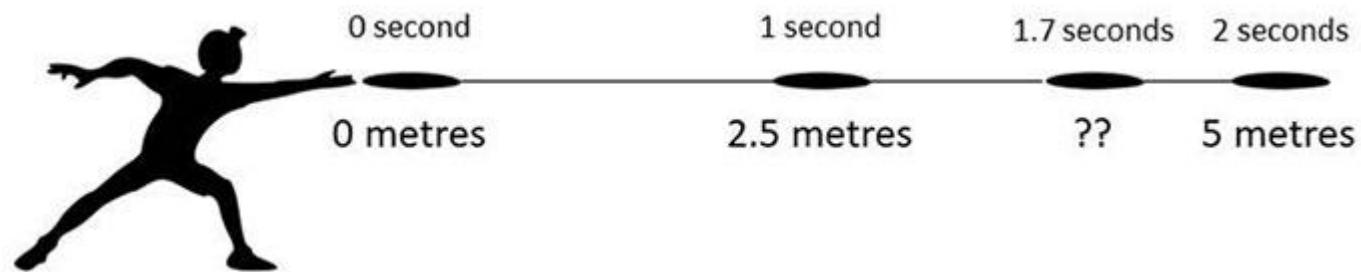
- ❑ Các tình huống khiến framerate không ổn định
- ❑ Quá nhiều GameObject trên màn hình
- ❑ Tính toán quá phức tạp hoặc hình ảnh quá lớn khiến CPU / GPU quá tải
- ❑ FPS thấp do cấu hình thiết bị không phù hợp

☐ Các chế độ **Interpolation**

- ☐ **None:** Không áp dụng Interpolation . Rigidbody chỉ cập nhật vị trí của nó khi Unity gọi `FixedUpdate()`, điều này có thể gây ra sự giật khung hình nếu tốc độ khung hình thay đổi.



- ❑ **Interpolate:** Trong chế độ này, Unity sử dụng dữ liệu từ khung hình trước đó để tính toán vị trí hiện tại của Rigidbody. Điều này giúp chuyển động trông mượt hơn khi tốc độ khung hình không đều.



❑ **Extrapolate** : Chế độ này ước tính vị trí tiếp theo của Rigidbody dựa trên tốc độ hiện tại của nó. Extrapolate được sử dụng khi bạn muốn dự đoán vị trí tiếp theo của đối tượng, và phù hợp khi đối tượng có tốc độ di chuyển cao.



- Điều chỉnh sử dụng linh hoạt trong câu lệnh khi xảy ra tình huống với FrameRate giúp chuyển động mượt hơn

```
20 void Start()  
21 {  
22     Rigidbody rb = GetComponent<Rigidbody>();  
23     // Bật chế độ Interpolation  
24     rb.interpolation = RigidbodyInterpolation.Interpolate;  
25     rb.interpolation = RigidbodyInterpolation.Extrapolate;  
26     rb.interpolation = RigidbodyInterpolation.None;
```

- ☐ Cách phát hiện FrameRate hoạt động không ổn định
- ☐ Sử dụng deltaTime
- ☐ Tính toán FPS
- ☐ Sử dụng Unity Profiler
- ☐ Sử dụng `Application.targetFrameRate`
- ☐ Kiểm tra `FixedUpdate()`

❑ Sử dụng deltaTime

```
void Update()
```

```
{
```

```
    Debug.Log("Delta Time: " + Time.deltaTime);
```

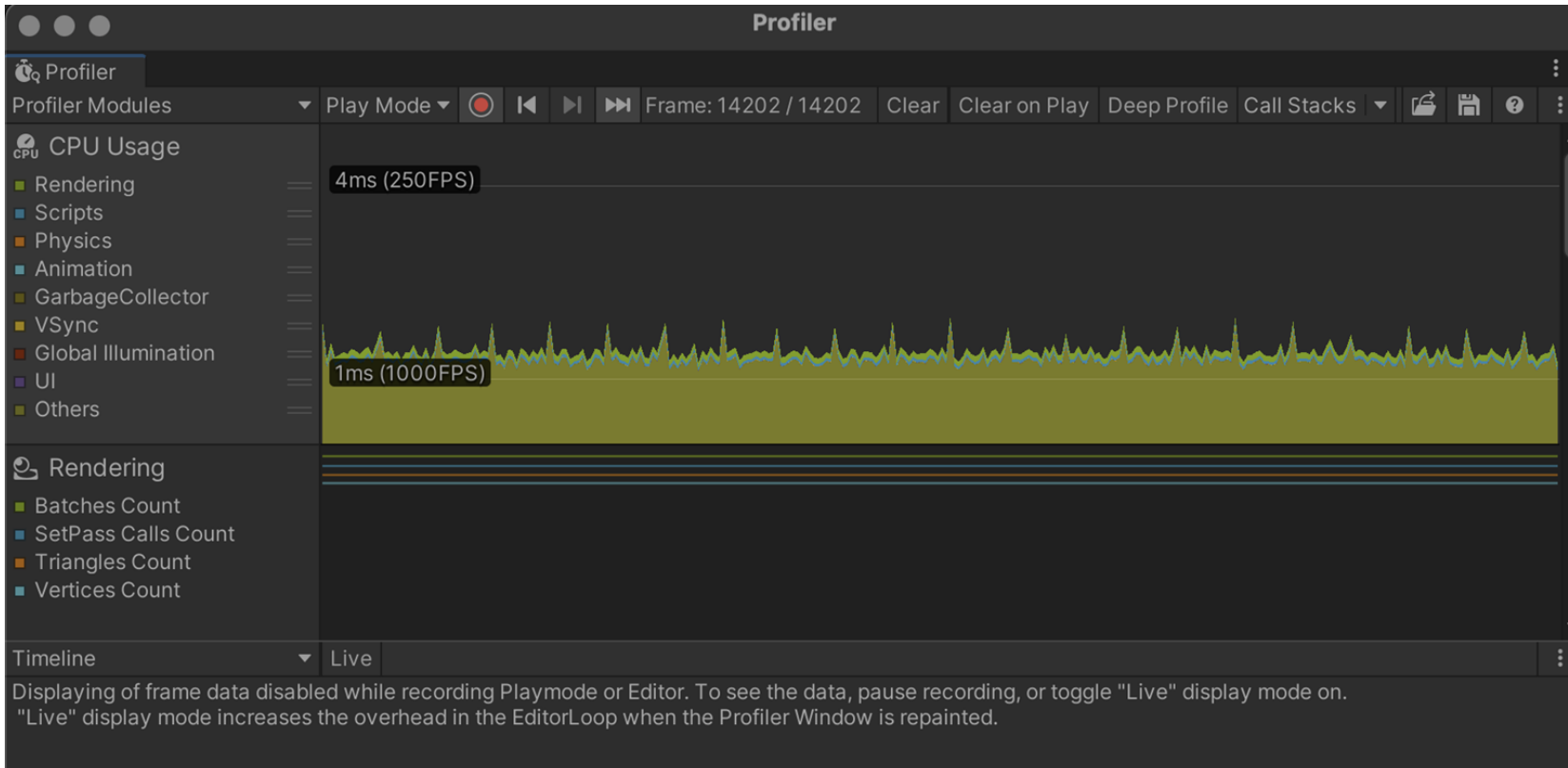
```
}
```

Nếu giá trị deltaTime thay đổi đột ngột, tức là
FrameRate đang thay đổi

- ❑ Tính toán FPS bằng cách hiển thị lên màn hình, nếu FPS thay đổi hoặc tăng giảm sâu tức là FPS không ổn định

```
31     private float deltaTime = 0.0f;  
    // Event function ? usages ? overrides ? extension methods ? exposing APIs  
32     void Update()  
33     {  
34         deltaTime += (Time.deltaTime - deltaTime) * 0.1f;  
35     }  
    // Event function ? usages ? overrides ? extension methods ? exposing APIs  
36     void OnGUI()  
37     {  
38         float fps = 1.0f / deltaTime;  
39         GUI.Label(position: new Rect(x: 10, y: 10, width: 100, height: 20), text: "FPS: " + Mathf.Ceil(fps));  
40     }
```

□ Sử dụng Unity Profiler : Windows -> Analytics -> Profiler



- ❑ Sử dụng `Application.targetFrameRate` :
- ❑ Thiết lập FPS bằng câu lệnh :
- ❑ `Application.targetFrameRate = 60;` // nếu chương trình không giữ được mức 60 fps tức là chương trình đang gặp vấn đề về xử lý

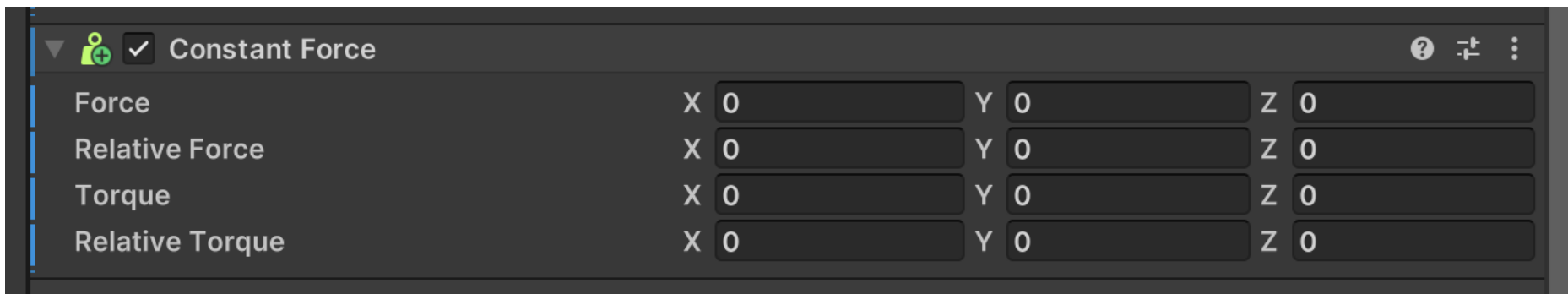
❑ Kiểm tra FixedUpdate() : theo dõi số lần

FixedUpdate được gọi để xác định xem FPS có bị ảnh hưởng bởi việc cập nhật vật lý hay không

```
31      private int fixedUpdateCount = 0;
      Event function ? usages ? overrides ? extension methods ? exposing APIs
32      void FixedUpdate()
33      {
34          fixedUpdateCount++;
35      }
      Event function ? usages ? overrides ? extension methods ? exposing APIs
36      void Update()
37      {
38          Debug.Log(message: "FixedUpdate Count: " + fixedUpdateCount);
39          fixedUpdateCount = 0;
40      }
```

- ❑ ConstantForce là một component dùng để áp dụng lực liên tục và mô-men xoắn (torque) lên một Rigidbody trong trò chơi.
- ❑ Dễ dàng tạo di chuyển hoặc xoay liên tục cho 1 GameObject mà không cần câu lệnh phức tạp

- ❑ ConstantForce là một component dùng để áp dụng lực liên tục và mô-men xoắn (torque) lên một Rigidbody trong trò chơi.
- ❑ Dễ dàng tạo di chuyển hoặc xoay liên tục cho 1 GameObject mà không cần câu lệnh phức tạp



- ❑ **force**: Lực tác dụng lên rigidbody mỗi khung hình.
- ❑ **relativeForce**: Lực tác dụng, dựa trên hệ tọa độ của rigidbody, mỗi khung hình.
- ❑ **relativeTorque**: Mô-men xoắn, dựa trên hệ tọa độ của rigidbody, tác dụng mỗi khung hình.
- ❑ **torque**: Mô-men xoắn tác dụng lên rigidbody mỗi khung hình.

□ Sử dụng thông qua câu lệnh

```
20 void Start()  
21 {  
22     ConstantForce constantForce = gameObject.AddComponent<ConstantForce>();  
23  
24     // Áp dụng lực liên tục về phía X  
25     constantForce.force = new Vector3(x:5, y:0, z:0);  
26  
27     // Áp dụng mô-men xoắn liên tục quanh trục Y  
28     constantForce.relativeTorque = new Vector3(x:0, y:10, z:0);  
29 }
```

- ❑ Lưu ý : Không sử dụng hoặc cân nhắc dùng quá nhiều ConstantForce
- ❑ ConstantForce khiến chương trình phải xử lý nhiều hơn do câu lệnh được áp dụng mỗi khi Frame được gọi



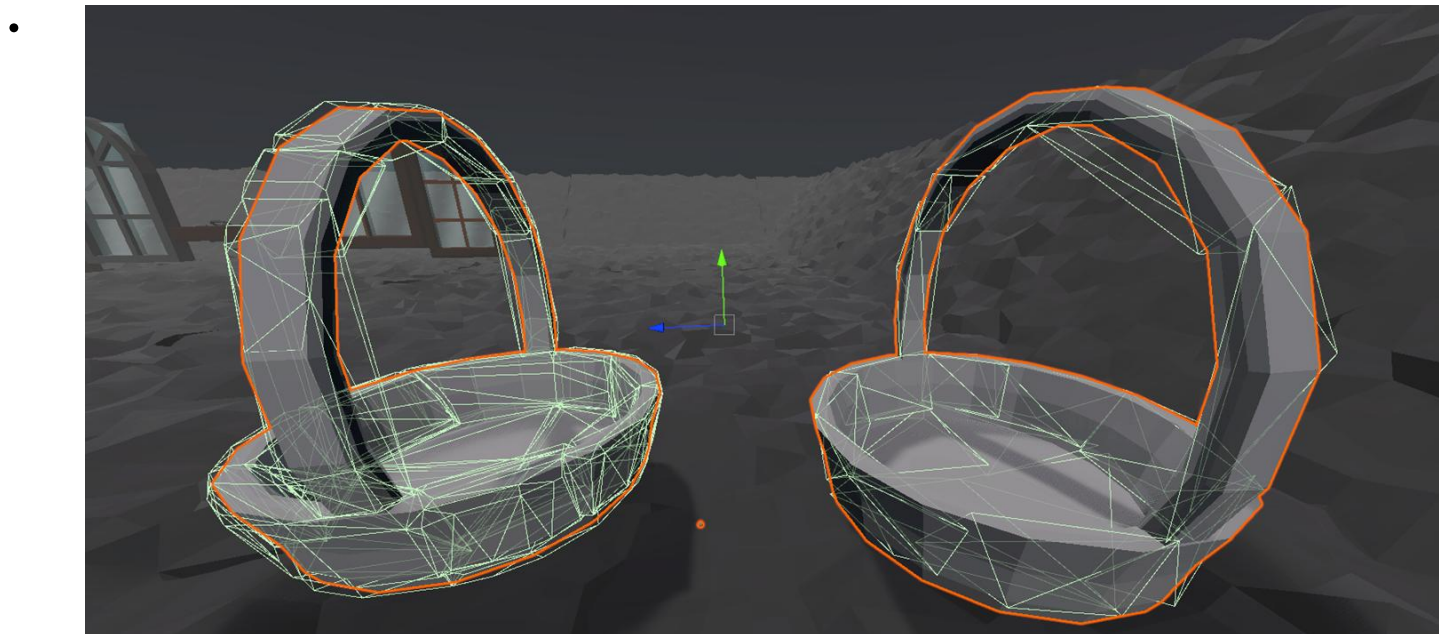
LẬP TRÌNH GAME 3D NÂNG CAO

BÀI 2: MeshCollider và Compound Collider
trong UNITY 3D

- ☐ MeshCollider trong Unity
- ☐ Concave Mesh Collider
- ☐ Convex Mesh collider
- ☐ Compound Collider trong Unity

- ❑ Mesh Collider là một loại collider sử dụng hình dạng của mesh để xác định vùng va chạm cho đối tượng trong Unity.
- ❑ Khác với các colliders hình học (Box, Sphere), Mesh Collider cho phép mô phỏng va chạm phức tạp hơn, phù hợp với các đối tượng có hình dạng không đều.

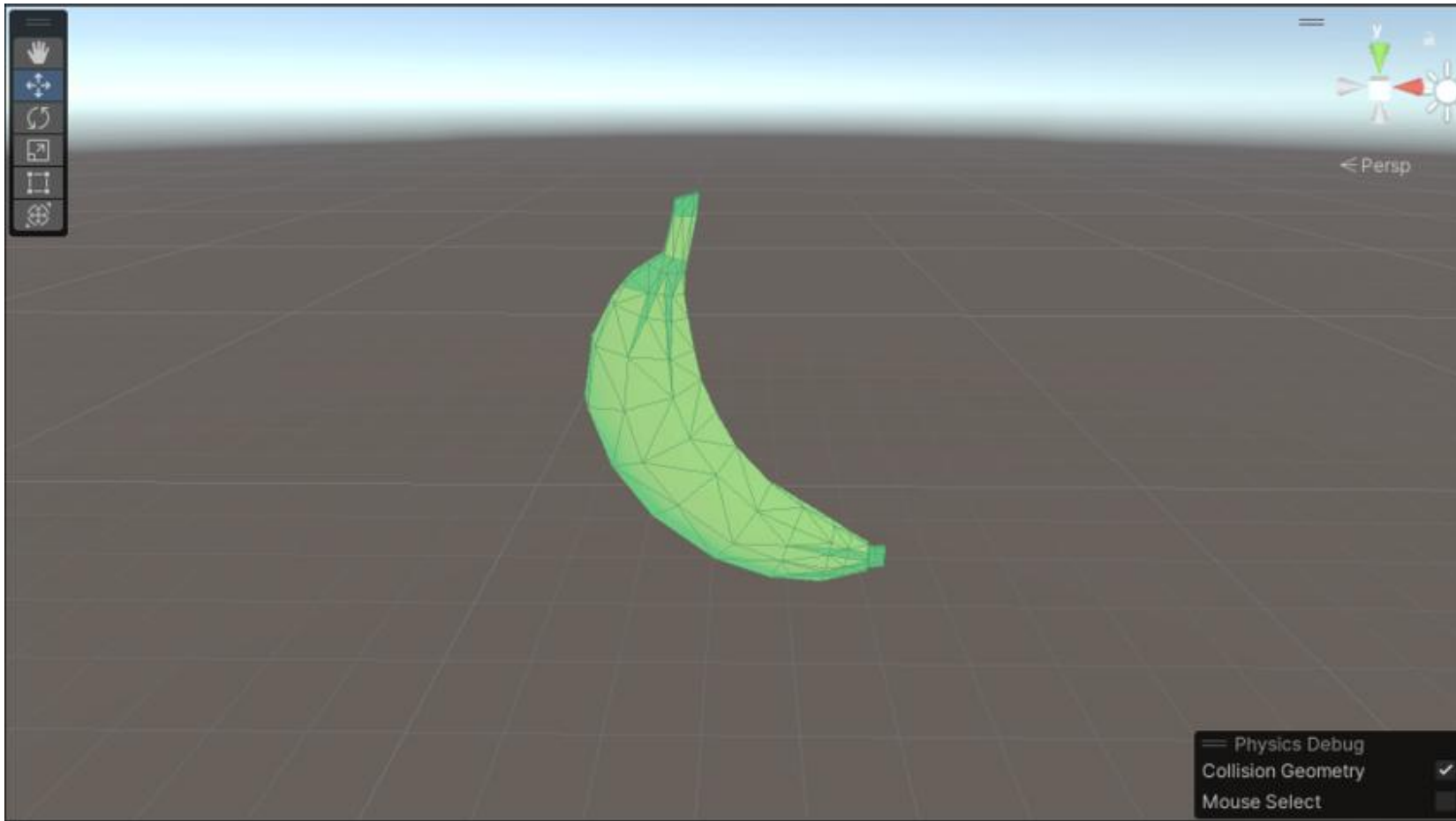
□ 2 loại Mesh Collider là concave và convex



- ❑ Concave (lỗm) nghĩa là collider sẽ bao quanh chính xác hình dạng của đối tượng, bao gồm cả các vùng lỗm, chi tiết và các chỗ hốc.
- ❑ Với đối tượng có hình quả chuối, concave Mesh Collider sẽ bọc sát theo hình dạng đặc thù của quả chuối, bao gồm cả phần lỗm bên trong, làm cho collider có thể mô phỏng va chạm rất chính xác.

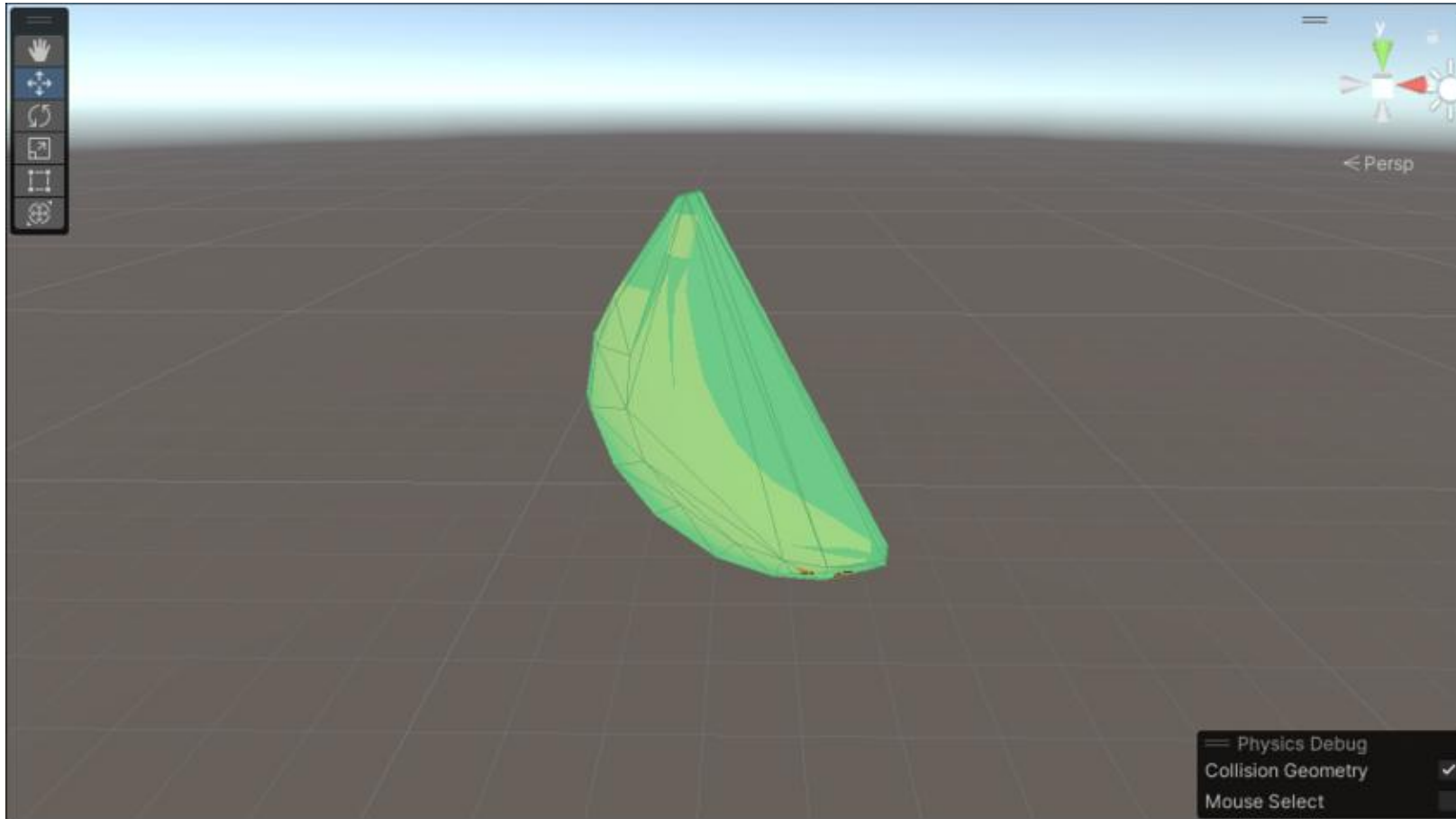
- ❑ Nhược điểm: Concave Mesh Collider tiêu tốn nhiều tài nguyên vì nó đòi hỏi nhiều tính toán hơn để xác định va chạm. Ngoài ra, dạng collider này không thể sử dụng với Rigidbody di chuyển, vì có thể gây vấn đề về hiệu suất và tính ổn định trong vật lý của Unity.

Concave Mesh Collider



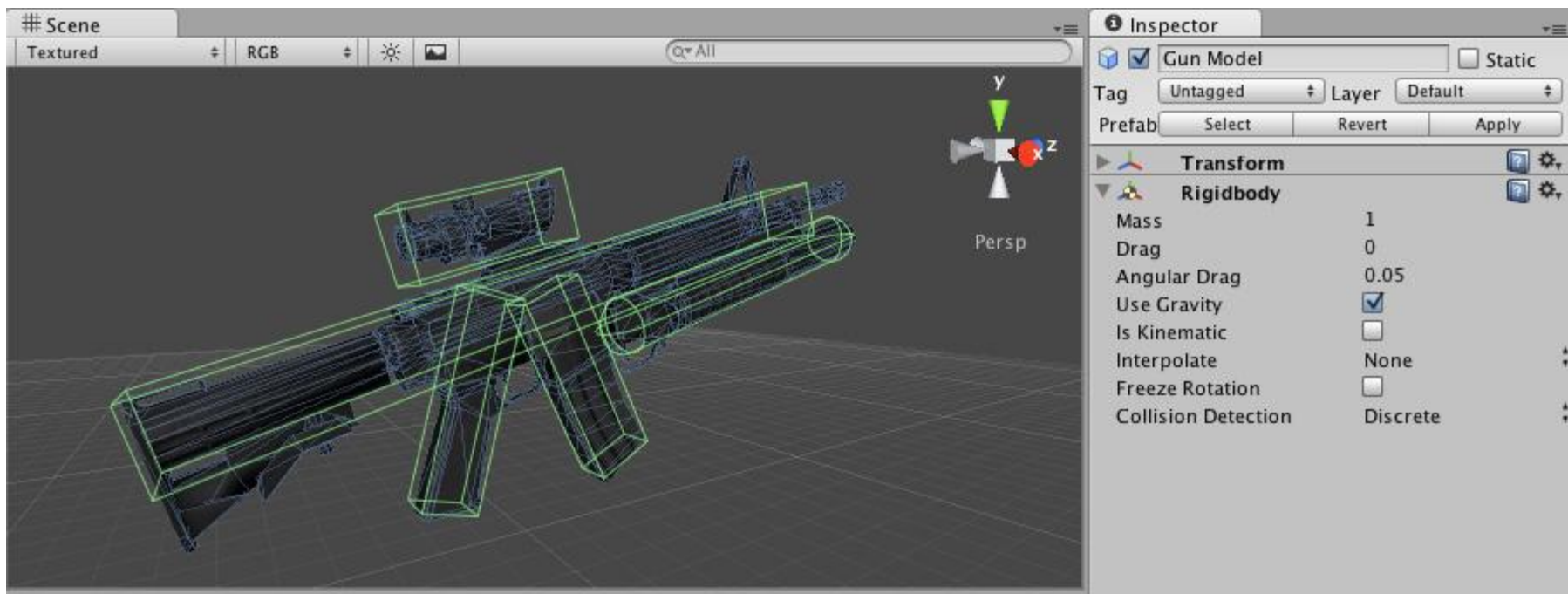
- Convex (lồi) nghĩa là collider sẽ tạo một lớp bao lồi xung quanh đối tượng, bỏ qua các vùng lõm hoặc các chi tiết nhỏ. Collider sẽ tạo một khối lồi đơn giản bao quanh toàn bộ đối tượng, sao cho bất kỳ đoạn thẳng nào nối hai điểm trong khối đều nằm hoàn toàn trong collider.

- **Convex Mesh Collider** sẽ bỏ qua phần lõm của quả chuối và tạo thành một hình lồi bao quanh, khiến nó trông giống như một khối tổng quát hình quả chuối.
- **Convex Mesh Collider** ít tốn tài nguyên hơn và có thể dùng với **Rigidbody** di chuyển, do Unity dễ dàng tính toán va chạm trên một bề mặt lồi.

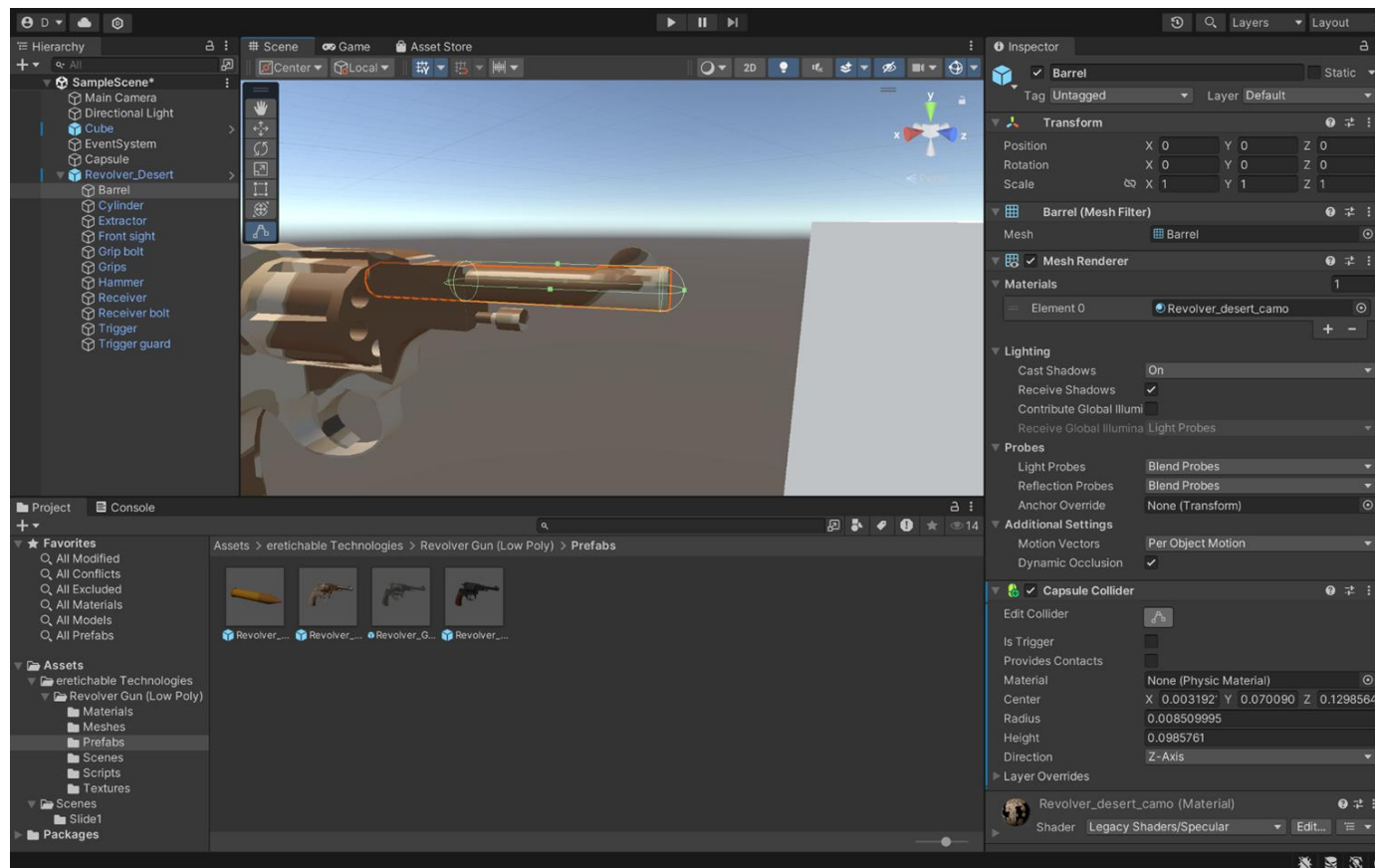


□ **Compound Colliders** là kỹ thuật trong Unity dùng để tạo ra vùng va chạm phức tạp cho các **GameObject** bằng cách kết hợp nhiều colliders đơn giản (như Box, Sphere, Capsule) thay vì sử dụng một **Mesh Collider** duy nhất

- ❑ Sử dụng kết hợp nhiều Collider thay vì Mesh Collider, giảm số lượng “đỉnh” phải tính toán mà vẫn đạt kết quả mong muốn



□ Bổ sung các Collider đơn giản vào 1 GameObject



- ☐ MeshCollider trong Unity
- ☐ Concave Mesh Collider
- ☐ Convex Mesh collider
- ☐ Compound Collider trong Unity
- ☐ GV giao bài tập



Kết thúc